Applied Soft Computing 52 (2017) 795–804

Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc

A local search based approach for solving the Travelling Thief Problem: The pros and cons

Mohamed El Yafrani*, Belaïd Ahiod

LRIT, Associated Unit to CNRST (URAC 29), Faculty of Science, Mohammed V University in Rabat, B.P. 1014 Rabat, Morocco

ARTICLE INFO

Article history:
Received 4 April 2015
Received in revised form 5 May 2016
Accepted 29 September 2016
Available online 3 October 2016

Keywords:
Interdependence
Metaheuristics
Local search
Combinatorial optimization
Travelling Thief Problem

ABSTRACT

The Travelling Thief Problem (TTP) is a novel problem that aims to provide a benchmark model of combinatorial optimization problems with multiple interdependent components. The TTP combines two other well known benchmark problems: the Travelling Salesman Problem (TSP) and the Knapsack Problem (KP). The aim of this paper is to study the interdependence between the TTP's components, and how it makes solving each sub-problem independently from the other useless for solving the overall problem. A local search approach is proposed to solve the TTP. Two simple iterative neighborhood algorithms based on our approach are presented, analyzed, and compared to other algorithms. Initialization strategies are empirically investigated. The experimental results confirm that our approach was able to find new better solutions for many TTP instances.

© 2016 Elsevier B.V. All rights reserved.

1. Introduction

In combinatorial optimization, scientists have always been interested in solving benchmark problems. These problems are known to be NP-hard [1] and many of them are based on/have real-world applications. In many situations they have direct applications, and in many others they act as a component of an overall problem. Such problems include, among many others, the Knapsack Problem [2] which has applications in decision making processes; the Travelling Salesman Problem [3] which naturally finds applications in logistics, transportation, and PCB design; and the Graph Coloring Problems [4] which find applications in compiler optimization and scheduling problems. These examples are often used to test the performance of metaheuristics [5], and over the last decades many algorithms were proposed to solve them.

Nevertheless, such benchmark problems do not usually provide a good model that efficiently represent real-world problems. In many real-world applications, optimization problems are composed of two or many sub-problems that are dependent on each others [6,7].

For instance, one of the most important applications of the TSP is the design of Printed Circuit Boards in Very Large Scale Integration (VLSI). The simple version of the problem consists of finding the least wasteful way of wiring the electronic components. This simply means that the length of the wire should be minimized. Thus, it is a direct TSP application.

However, the VLSI optimization process is a more complicated task. In some practical situations, the wiring track should not cross some sensitive chips, as explained in [8]. Due to this constraint, the problem consists of two NP-hard sub-problems: First, wire some initially placed chips such as the wiring length is minimal. Second, change the chip's location such as the wiring track does not cross the sensitive ones. The interdependence between these two sub-problems is obvious. Given an initial least wasteful wiring plan, the placement clearly affects it by moving the chips, i.e. it mostly lowers the quality of the wiring plan. Correspondingly, given an initial placement plan such as there is no crossing between sensitive chips and the wiring track, the establishment of a new wiring plan may lead to crossing problems. This problem is somehow close to the Location-Routing Problem [9].

The above example demonstrates that in similar real-world situations it is pointless to consider solving sub-problems in isolation without considering the interdependence. Solving such problems is a difficult task that requires designing appropriate algorithms in an overall way, instead of tackling sub-problems individually. It also shows the gap between benchmark problems, which are mostly single-component, and real-world problems, which consist of multiple interdependent components.

The Travelling Thief Problem (TTP) [8] is a novel problem introduced with the aim of reducing the gap between real-world

* Corresponding author.
E-mail address: m.elyafrani@gmail.com (M. El Yafrani).

http://dx.doi.org/10.1016/j.asoc.2016.09.047
1568-4946/© 2016 Elsevier B.V. All rights reserved.

and benchmark problems. The TTP provides a NP-hard problem with multiple interdependent sub-problems. Such model will facilitate algorithm design for multi-component problems and benchmark testing. The TTP combines two combinatorial optimization problems: the Travelling Salesman Problem (TSP) and the Knapsack Problem (KP).

Since the introduction of the TTP, some algorithms were proposed to solve it. In [10], the authors proposed three heuristics, a simple constructive heuristic and two iterative algorithms (Evolutionary Algorithm and Random Local Search). The authors aim to deliver a first set of heuristics that can solve efficiently the TTP. In [11], another iterative approach was proposed to solve the problem more efficiently. The approach introduced in this paper is designed particularly for problems with multiple components. The idea is to manage the communication between sub-problems to optimize the solution. Another approach consists on adapting the Cooperative Coevolution metaheuristic to solve the TTP [12]. A Memetic Algorithm was also introduced in the same paper. In [13], the same authors proposed a Memetic Algorithm able to solve very large TTP instances.

Many real-world multi-component problems (also referred to as multi-silo problems [14,15]) have been already investigated by the Operations Research community. Most of these problems occur in the context of supply chain management [16–18]. Other multi-component problems include the vehicle routing problems under loading constraint [19], the vehicle routing problem under packing and bundling constraints [20], and the pencil puzzle killer Sudoku in its generalized form (i.e. $N^2 \times N^2$ grid with $N \times N$ regions) which combines elements of Sudoku [21] with some Kakuro [22] rules.

These problems are composed of many interdependent components that are usually NP-hard by themselves. Thereby, they are naturally more intractable than academic problems such as TSP and KP. Therefore, metaheuristics and soft computing techniques are a good fit to solve this family of problems. These approaches are widely used for such intractable problems. Indeed, soft computing/metaheuristic approaches such as Simulated Annealing, Particle Swarm Optimization, and Tabu Search were proposed to tackle multi-component problems [23–27].

In this paper, we discuss the pros and cons of a local search implementation to solve the TTP. We aim to analyze the issues of implementing local search algorithms to solve a multi-component problem. We also propose an approach to implement neighborhood deterministic local search algorithms to solve small and mid-size TTP instances. Our approach is based on neighborhood combination and the tracking of some solution's parameters for evaluating mutated solutions.

The rest of this article is organized as follows. In Section 2, a background about TSP and KP is given and the TTP is introduced. In Section 3, the interdependence in the TTP is discussed and a local search approach is proposed. The proposed algorithms are applied on some TTP instances and the experimental results are discussed in Section 4. Finally, Section 5 concludes the paper and outlines some future directions.

2. The Travelling Thief Problem

In this section, we formulate the Travelling Thief Problem (TTP)[1] which combines TSP and KP. We also investigate the interdependence between these two sub-problems in the TTP, and explain the difficulties of implementing a neighborhood based algorithm.

2.1. Problem formulation

In the TTP, we consider n cities and the associated distance matrix $\{d_{ij}\}$. There are m items scattered in these cities, each item k having a profit p_k and a weight w_k. A thief is going to visit all these cities only once and pick up some items to fill his knapsack. W denotes the maximum capacity of the knapsack, $v_{\min}$ and $v_{\max}$ are the minimum and maximum possible velocity respectively. We also consider the following constraints and parameters:

- Each item is available in only one city.[2] We note $\{A_i\}$ the availability vector, $A_i \in \{1, \ldots, n\}$ contains the reference to the city that contains the item i.
- The knapsack is rented. We note R the renting price per time unit.
- The velocity of the thief changes accordingly to the knapsack weight. The more the knapsack gets heavy, the more the thief gets slow. We note v_{x_i} the velocity of the thief at city x_i, v_{x_i} is defined in Eq. (1).

$$v_{x_i} = v_{\max} - C * w_{x_i} \tag{1}$$

where $C = \frac{v_{\max} - v_{\min}}{W}$ is a constant value, and w_{x_i} represents the weight of the knapsack at city x_i.

The goal of the problem is to find the tour x and the picking plan z that maximize the total travel gain defined in Eq. (2).

$$G(x, z) = g(z) - R * f(x, z) \tag{2}$$

where $g(z) = \sum_m p_m * z_m$ is the total value of the items subject to $\sum_m w_m * z_m \le W$, and $f(x, z) = \sum_{i=1}^{n-1} t_{x_i, x_{i+1}} + t_{x_n, x_1}$ is the total travel time.

Additionally, the solutions are coded as follows:

- $x = (x_1, \ldots, x_n)$ is a vector containing the ordered list of cities.
- $z = (z_1, \ldots, z_n)$ is a binary vector, z_i is equal to 1 if the item i is picked, 0 otherwise.

2.2. Analysis of interdependence

In single-component problems such as TSP and KP, it is usually easy to implement a neighborhood-based algorithm such as the 2-OPT search [28], a bit-flip search, or a Simulated Annealing [29]. The reason for this is that the objective value of neighbors of a given solution is "easily" deduced from the objective value of the original solution.

To illustrate this, we use a simple example in the context of a symmetric TSP problem (TSP is said symmetric if $d_{ij} = d_{ji}$ for $1 \le i, j \le n$). Consider the objective function f, the distance matrix $\{d_{ij}\}$, a tour $x = (x_1, \ldots, x_n)$, and a neighbor tour x^i which is obtained by swapping two adjacent cities x_i and x_{i+1}: $x^i = (x_1, \ldots, x_{i+1}, x_i, \ldots, x_n)$. Therefore, we can compute the objective value of x^i in a constant time as shown in Eqs. (3)–(5).

$$f(x) = d_{x_1,x_2} + \cdots + d_{x_{i-1},x_i} + d_{x_i,x_{i+1}} + d_{x_{i+1},x_{i+2}} + \cdots + d_{x_n,x_1} \tag{3}$$

$$f(x^i) = d_{x_1,x_2} + \cdots + d_{x_{i-1},x_{i+1}} + d_{x_{i+1},x_i} + d_{x_{i-1},x_i} + \cdots + d_{x_n,x_1} \tag{4}$$

[1] In this paper we study the TTP1 which is a mono-objective version of the TTP as introduced in [8].

[2] This is different from the TTP1 as introduced in [8] in which an item can be found in many cities. The reason to this is due to the data generator introduced in [10].

$$f(x^i) = f(x) + \Delta = f(x) + d_{x_{i-1},x_i} + d_{x_{i+1},x_{i+2}} - (d_{x_{i-1},x_{i+1}} + d_{x_{i-1},x_i}) \tag{5}$$

In Eqs. (3) and (4), we see clearly that the change affects only the terms around the region of change in the objective function. This gives us the possibility to compute easily the difference between the two objective values. Thus, this difference can be used to compute the objective value of the neighbor. In the rest of this paper, we will refer to this technique as the *delta technique*.

However, in a multi-component problem such as the TTP, the combination leads to interdependence between the components. In other words, this means that, given a TTP solution, when a change occurs in the tour or in the picking plan it affects the objective value of the solution in a way that cannot be "easily" predicted.

Note that to compute the TTP objective function G, we need to evaluate the total value of items g and the total travel time f. Computing g is a very simple task using the delta technique explained above. Whereas the formula of f is very complex as it is a non-linear two variables function.

To investigate this behavior in detail, we consider a TTP solution (x, z) such as $x = x_1, \ldots, x_n$ is the tour, $z = z_1, \ldots, z_n$ is the picking plan, and $f(x, z)$ is the total traveling time defined in Eq. (6).

$$f(x,z) = \frac{d_{x_1,x_2}}{v_{\max} - C * w_{x_1}} + \cdots + \frac{d_{x_{i-1},x_i}}{v_{\max} - C * w_{x_{i-1}}} + \frac{d_{x_i,x_{i+1}}}{v_{\max} - C * w_{x_i}} + \frac{d_{x_{i+1},x_{i+2}}}{v_{\max} - C * w_{x_{i+1}}} + \cdots + \frac{d_{x_n,x_1}}{v_{\max} - C * w_{x_n}} \tag{6}$$

where w_{x_i} is the knapsack weight at city x_i.

Let us consider also the two simple cases below.

2.2.1. Case 1: swapping two adjacent cities

Let (x^i, z) be the neighbor solution of (x, z) such as the nodes x_i and x_{i+1} are swapped.

Computing the total traveling time for (x^i, z) is simple using the following Eq. (7):

$$f(x^i,z) = \frac{d_{x_1,x_2}}{v_{\max} - C * w_{x_1}} + \cdots + \frac{d_{x_{i-1},x_{i+1}}}{v_{\max} - C * w_{x_{i-1}}} + \frac{d_{x_i,x_{i+1}}}{v_{\max} - C * (w_{x_{i-1}} + w_{x_{i+1}} - w_{x_i})} + \frac{d_{x_{i+1},x_{i+2}}}{v_{\max} - C * w_{x_{i+1}}} + \cdots + \frac{d_{x_n,x_1}}{v_{\max} - C * w_{x_n}} \tag{7}$$

As demonstrated in Eq. (7), the swap affects only the region around i as in TSP, which makes it possible to recover total time for (x^i, z) using basic operations and by knowing the knapsack weight at cities x_{i-1} and x_{i+1}.

2.2.2. Case 2: picking/unpacking an item

In this second case, we consider the solution (x, z^k) such as the item z_k is picked (or unpacked) from city i. In this case it is more complicated to recover the total traveling time:

$$f(x,z^k) = \frac{d_{x_1,x_2}}{v_{\max} - C * w_{x_1}} + \cdots + \frac{d_{x_{i-1},x_i}}{v_{\max} - C * w_{x_{i-1}}} + \frac{d_{x_i,x_{i+1}}}{v_{\max} - C * (w_{x_i} + \Delta_w)} + \frac{d_{x_{i+1},x_{i+2}}}{v_{\max} - C * (w_{x_{i+1}} + \Delta_w)} + \cdots + \frac{d_{x_n,x_1}}{v_{\max} - C * (w_{x_n} + \Delta_w)} \tag{8}$$

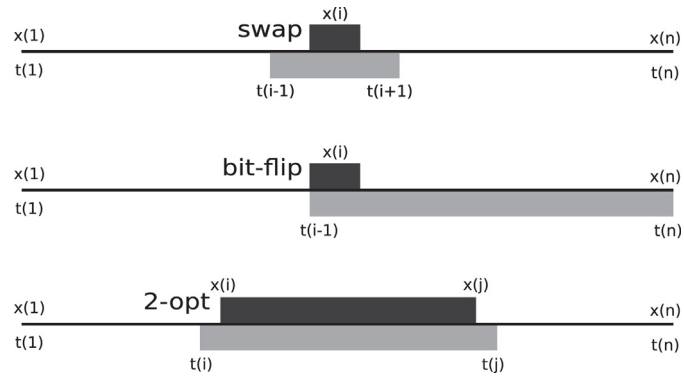

Fig. 1. Example of changes that can be applied to a TTP solution (in black) and its effect on the total traveling time function (in gray).

Eq. (8) shows clearly that it is impossible to recover the traveling time in a constant time. Indeed, the change affects all the terms that come after the region of change. Therefore, the value $f(x, z^k)$ can be recovered partially from $f(x, z)$ by reusing the terms before the region of change, and keeping track of the time and weight of the original solution.

Fig. 1 illustrates this behavior by showing the change region in the tour (in black) and its corresponding effect in the total traveling time value (in gray). Note that $x(i)$ represents the city at index i and $t(i) = \frac{d_{x_i,x_{i+1}}}{v_{\max} - C * w_{x_i}}$ is the ith term of the total time function f. Three cases are illustrated: a simple swap of two adjacent cities, a bit-flip (the corresponding city is illustrated), and a 2-OPT inversion. The effect here means the change that occurs in the terms of the total traveling time function f. In other words, the effect represents the terms in the objective function that cannot be reused to calculate the neighbor's objective value.

3. Proposed approach

In this section we present our approach for solving the TTP using local search algorithms. Our approach uses neighborhood algorithms and proposes a set of techniques to simplify the recovery of the objective value. The proposed approach is based on two ideas. First, joining TSP and KP's neighborhoods; and second, tracking some solution's parameters to calculate the objective value of mutated solutions.

In this paper we present two simple deterministic algorithms. The first combines TSP's N1 neighborhood (swapping two adjacent cities) and KP's one-bit-flip (changing the state of an item). And the second is a combination of TSP's 2-OPT (inversion of a sub-tour) neighborhood and KP's one-bit-flip.

3.1. JNB: Joint N1-BF

JNB (standing for Joint N1-BF) is a neighborhood-based heuristic that combines the TSP's N1 neighborhood and KP's one-bit-flip. N1 consists of swapping two adjacent cities. Whereas one-bit-flip consists of toggling the state of an item, i.e. picking the item if it is not picked and unpacking it otherwise.

JNB consists of applying a Cartesian product between these two neighborhoods to generate a neighborhood specific to the TTP. Then the generated neighborhood is browsed in order to select a better (or the better) solution of the set.

As in other neighborhood based algorithms, two versions can be implemented using the same neighborhood. The first version is the hill climbing algorithm (will be referred to as first-fit), which stops browsing the neighborhood when a better solution is found.

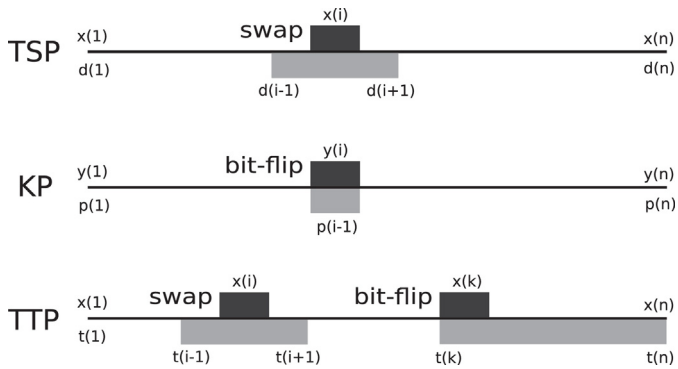

Fig. 2. Swap applied to a TTP solution and its effect on the total traveling time function, compared to changes in TSP and KP.

The second version is the steepest descent (will be referred to as best-fit), which browses the whole neighborhood in order to select the best solution.

As explained in Section 2, it is practically impossible to calculate the total traveling time of a neighbor solution in a constant time when a bit is flipped in the picking plan. The best that can be done is to recalculate the value of f starting from the city where the bit-flip happened.

Fig. 2 illustrates how the objective value of a neighbor solution is recovered in TTP using the JNB algorithm, and a comparison to equivalent operations for KP and TSP. Note that, in the context of KP, the bit-flip region (in black) corresponds to the item's index in the picking plan. Whereas in the TTP, it represents the index of the city from which the item is picked. In addition, $d(i)$ represents the distance between the cities $x(i)$ and $x(i+1)$, $y(i)$ is the ith item, and $p(i)$ is the profit of the ith item.

The recovery operation also needs to keep track of the weight and time accumulation at each city of the tour. For this purpose, we propose three vectors to associate with each TTP solution:

- **Time accumulator** (t^{acc}): a vector that contains the current time at each city of the tour.
- **Weight accumulator** (w^{acc}): a vector that contains the current weight at each city of the tour.
- **Tour mapper** (t^{map}): since each city has a unique integer reference between 1 and n, t^{map} represents a vector that associates the index (position of the city in the tour) to the reference of city. For instance, let x be a tour, the t^{map} associated to x is obtained as shown below:

index	:	1, 2, 3, 4, 5, 6
x	:	1, 3, 5, 4, 6, 2
t^{map}	:	1, 6, 2, 4, 3, 5

The tour mapper is useful to get the position where an item is picked easily instead of browsing tour cities.

These vectors are recalculated for each solution at each heuristic iteration. It would be convenient to associate them to the objective function.

Algorithm 1 starts by initializing the tour and picking plan in lines 1 and 2. In our implementation we always use a Lin–Kernighan [30] tour as the initial TTP tour. We use three different techniques to generate the initial picking plan: no items picked, greedy algorithm, and random picking plan. Once the solution is initialized, the objective function calculates the total travel gain G, the travel time t, the total items profit p, and the three vectors t^{acc}, w^{acc} and t^{map} introduced above are updated in lines 4 and 5. Then, a swap is applied between two adjacent cities of the tour to generate a neighbor solution, and the vectors $\overline{t^{acc}}$, $\overline{w^{acc}}$ and $\overline{t^{map}}$ associated to the solution $(\bar{x}, z)$ are recovered from the original solution, see lines 6–10. For each swapped tour, we associate a bit-flipped picking plan, we ensure that there is space left in the knapsack if an item is picked, and we recover the profit p using the delta technique in line 15. Then, the total time is partially recalculated starting from the position of bit-flip i_{BF} in lines 16–21. Finally, in lines 22–31, the new total gain is calculated and compared to the current gain, if it improves it we keep the new solution and the process is iterated, otherwise the algorithm stops and the current solution is saved.

Algorithm 1. JNB (first-fit version)

```
1   x ← initial tour                                  ▷ we always use a Lin–Kernighan tour
2   z ← initial picking plan
3   repeat
4     Evaluate G, t, and p for (x, z)
5     Update t^map, t^acc, and w^acc
6     for i ← 2 . . . n − 1 do
7       x̄ ← swap(x, i)
8
      t̄^map ← tour mapper for (x̄, z)
9       t̄^acc ←
      recover time accumulator for (x̄, z)
10      w̄^acc ←
      recover weight accumulator for (x̄, z)
11      for k ← 1 . . . m do
12        z̄ ← bitflip(z, k)
13        if
      no space left then skip iteration
      end if
14        calculate Δ_w and Δ_p
15        p̄ ← p + Δ_p
16        i_BF ← t^map_{A_k}                            ▷ index of bit-flip
17        if i_BF = 0 then t̄ ← 0 else
      t̄ ← t̄^acc_{i_BF−1} end if
18        for r ← i_BF . . . n do
19          w_c ← w̄^acc_r + Δ_w
20          t̄ ← t̄ + d_{x_r, x̄_{r+1 mod n}} / (v_max − w_c ∗ C)
21        end for
22        Ḡ ← p̄ − R ∗ t̄
23        if Ḡ > G then break loop                      ▷ first fit
      end if
24      end for
25      if Ḡ > G break loop end if                      ▷ first fit
26    end for
27    if Ḡ > G then
28      x ← x̄
29      z ← z̄
30    end if
31  Until Ḡ ≤ G
```

Therefore, the worst-case complexity (i.e. bit-flips start in the first city) for one iteration of JNB algorithm is as follows:

$$O(JNB) = O(evaluation) + O(update) + O(swap) * O(bitflip) * O(backup) = O(m*n) + O(m*n) + O(n) * O(m) * O(n) = O(m*n^2) \quad (9)$$

where

- $O(evaluation)$ is the complexity of evaluating a TTP solution (line 4).
- $O(update)$ is the complexity of updating the associated vectors (line 5).
- $O(swap)$ is the swap's complexity (line 6).
- $O(bitflip)$ is the bitflip's complexity (line 11).

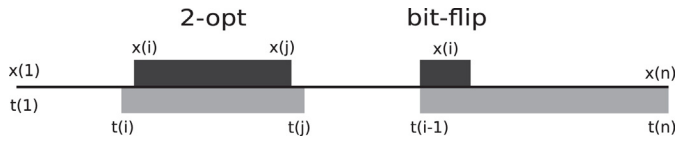

Fig. 3. 2-OPT applied to a TTP solution and its effect on the total traveling time function.

- $O(backup)$ is the complexity of the backup operation applied to the neighbor solution to retrieve the travel time (line 18).

As a result of interdependence between picking and travelling, the backup operation is introduced to partially recalculate the travel time. This operation has a linear worst-case complexity. Nevertheless, the recovery starts from the city of bit-flip instead of the first city which make the algorithm faster.

3.2. J2B: Joint 2opt-BF

J2B (standing for Joint 2opt-BF) is a neighborhood-based heuristic based on the same technique proposed in the previous subsection. This heuristic combines the TSP's 2-OPT neighborhood and KP's one-bit-flip. 2-OPT is an operator that consists of inverting a sub-tour of a given TSP tour.

As in JNB, a Cartesian product is applied between these two neighborhoods to generate a neighborhood specific to the TTP. Then, the generated neighborhood is browsed in order to select a better (or the better) solution of the set. Thereby, the worst-case complexity of this algorithm is $O(m * n^3)$.

The process of recovering the objective value is illustrated in Fig. 3. The figure shows that the recovery is more time complex as the 2-OPT (applied between x_i and x_j) affects all the terms associated to the cities between x_i and x_j. The reason for this is that the non-symmetry of the time function, i.e. $t(c_1, c_2) \neq t(c_2, c_1)$ such as t is the time function, and c_1, c_2 are two given cities.

In this heuristic, two more history vectors are needed:

- **Time register** (t^{reg}): a vector that contains the added time at each city of the tour.
- **Weight register** (w^{reg}): a vector that contains the added weight at each city of the tour.

Algorithm 2 starts by initializing the tour and picking plan using the same technique as in JNB in lines 1 and 2. Once the solution is initialized, the objective function calculates the travel gain G, the travel time t, the total items profit p, t^{map} and the four history vectors introduced above in lines 4 and 5. Then, a 2-OPT is applied between two cities of the tour to generate a neighbor solution and t^{map}. The history vectors are also recovered for this solution in lines 6–13. For each obtained tour, we associate a bit-flipped picking plan, and the w^{rec} vector is recovered for the new solution. We ensure that there is space left in the knapsack if an item is picked. Therefore, we recover the profit p using the *delta technique* in lines 14–19. Then, the total time is partially recalculated starting from the position of bit-flip i_{BF} in lines 20–26. Finally, in lines 27–38, the new total gain is calculated and compared to the current gain. If the new gain improves the current one, we keep the new solution and the process is iterated. Otherwise, the algorithm stops and the current solution are returned.

Algorithm 2. J2B (first-fit version)

1	$x \leftarrow$ *initial tour*	▷ we always use a Lin–Kernighan tour
2	$z \leftarrow$ *initial picking plan*	
3	**repeat**	
4	Evaluate G, t, and p for (x, z)	
5	Update t^{map}, t^{acc}, w^{acc}, t^{rec}, and w^{rec}	
6	**for** $i \leftarrow 2 \ldots n-1$ **do**	
7	**for**$j \leftarrow i+1 \ldots n$ **do**	
8	$\bar{x} \leftarrow$ *do2opt*(x, i, j)	
9	$\overline{t^{map}} \leftarrow$ *tour mapper for* $(\bar{x}, z)$	
10	$\overline{t^{acc}} \leftarrow$ *recover time accumulator for* $(\bar{x}, z)$	
11	$\overline{w^{acc}} \leftarrow$ *recover weight accumulator for* $(\bar{x}, z)$	
12	$\overline{t^{rec}} \leftarrow$ *recover time register for* $(\bar{x}, z)$	
13	$\overline{w^{rec}} \leftarrow$ *recover weight register for* $(\bar{x}, z)$	
14	**for** $k \leftarrow 1 \ldots m$ **do**	
15	$\bar{z} \leftarrow$ *bitflip*(z, k)	
16	$\hat{w^{rec}} \leftarrow$ *recover weight register for* $(\bar{x}, \bar{z})$	
17	**if** *no space left* **then** *skip iteration* **end if**	
18	*calculate* Δ_w *and* Δ_p	
19	$\bar{p} \leftarrow p + \Delta_p$	
20	$i_{BF} \leftarrow t^{map}_{A_k}$	▷ index of bit-flip
21	**if** $i_{BF} = 0$ **then** $\bar{t} \leftarrow 0$ **else** $\bar{t} \leftarrow \overline{t^{acc}_{i_{BF}-1}}$ **end if**	
22	**if** $i_{BF} = 0$ **then** $w_c \leftarrow 0$ **else** $w_c \leftarrow \overline{w^{acc}_{i_{BF}-1}}$ **end if**	
23	**for** $r \leftarrow i_{BF} \ldots n$ **do**	
24	$w_c \leftarrow w_c + \hat{w^{acc}_r} + \Delta_w$	
25	$\bar{t} \leftarrow \bar{t} + \frac{d_{\bar{x}_r, \bar{x}_{r+1 modn}}}{v_{\max} - w_c * C}$	
26	**end for**	
27	$\bar{G} \leftarrow \bar{p} - R * \bar{t}$	
28	**if** $\bar{G} > G$ **then** *break loop* **end if**	▷ first fit
29	**end for**	
30	**if** $\bar{G} > G$ **then** *break loop* **end if**	▷ first fit
31	**end for**	
32	**if** $\bar{G} > G$ **then** *break loop* **end if**	▷ first fit
33	**end for**	
34	**if** $\bar{G} > G$ **then**	
35	$x \leftarrow \bar{x}$	
36	$z \leftarrow \bar{z}$	
37	**end if**	
38	**until** $\bar{G} \leq G$	

Therefore, the worst-case complexity (i.e. bit-flips start in the first city) for one iteration of J2B algorithm is as follows:

$$\begin{aligned} O(J2B) &= O(evaluation) + O(update) + O(2opt) * O(bitflip) \\ &* \ O(backup) = O(m * n) + O(m * n) + O(n^2) * O(m) * O(n) \\ &= O(m * n^3) \end{aligned} \tag{10}$$

where

- $O(2opt)$ is the 2-OPT complexity (line 6).

This algorithm has clearly a higher time complexity due to the use of 2-OPT neighborhood.

Note that the *do2opt* function at line 8 is used for the sake of algorithm readability. In practice, the swap is not applied until the solution is accepted. Instead of applying the 2-OPT exchange, the original solution is used to access neighbor's cities in a constant time using the function in Algorithm 3. The function has four parameters: k represents the required city index, x is the tour, i and j are the 2-OPT indices.

Algorithm 3. Access 2-OPT elements

```
function GET2OPT (k, x, i, j)
  if k ≥ i & k ≤ j then
    return x_j − k + i
  end if
  return x_k
end function
```

3.3. On the complexity of the algorithms

Both JNB and J2B have a high time complexity. This is mainly due to the Cartesian product between the TSP and KP neighborhoods. Therefore, the time complexity of the resulting neighborhood is the product of the two neighborhoods (see Eqs. (9) and (10)). However, the complexity of the 2opt-BF neighborhood can be reduced using the Delaunay triangulation. Indeed, in a Delaunay triangulation, each city has 6 surrounding cities on average, which reduces significantly the time complexity. Moreover, for a given city, one can consider only the direct neighbors, the neighbors of neighbors, or even the entire Delaunay set [31].

The objective function (see Eq. (2)) also has a high time complexity. However, its complexity can be easily reduced from $O(m \times n)$ to $O(k \times n)$, where k is the number of items per city. This can be done by arranging the items per cities in a key-value data structure. In this case, the key would be the city reference, and the value would be the list of items in that city.

Furthermore, another technique that can be used in order to obtain a better complexity is to combine the neighborhoods sequentially. Thereby, the resulting time complexity would be the sum of the complexities of the two neighborhoods [13].

4. Experimental results

In this section, we present our testing strategy and the obtained results in terms of runtime and solution quality. Our testing plan consists of comparisons of solution initialization approaches and solving heuristics.

First, we compare some solution initialization strategies. For this purpose we use two different tour initialization strategies and three picking plan initialization strategies. The aim of these tests is to see how much the initial solution influences the algorithm designed to solve the TTP (in this context, local search algorithm). Note that for strategies with random behavior we run 10 independent executions and use the mean values for comparison.

Second, we compare our two algorithms[3] alongside two other heuristics proposed in [10]. The tests are performed on TTP instances introduced in the same paper.[4] The algorithms have a maximum runtime limit of 600 s. Once the runtime exceeds the limit, the algorithm stops and returns the last found solution.

Note that our implementation uses the Java platform and the tests are performed on a core i3-2370M CPU 2.40 GHz, with 4 GB of RAM, running Linux.

4.1. Initialization approaches

4.1.1. Initial tour

We compare two tour initialization methods. The first approach uses a naive random algorithm to generate a tour. The second uses the Lin–Kernighan heuristic to find a good TSP tour.

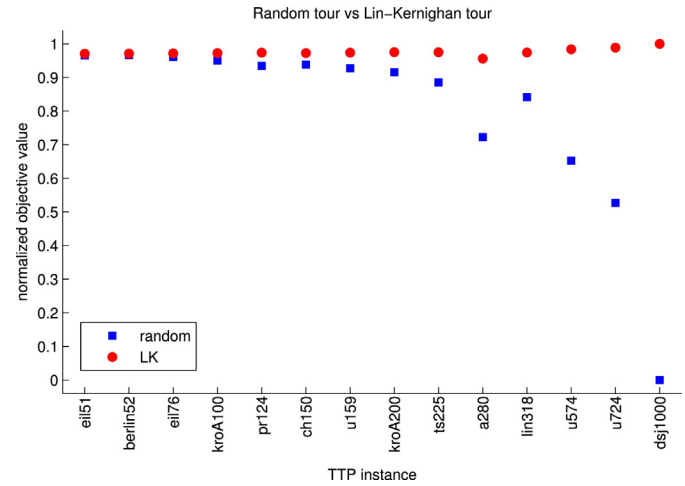

Fig. 4. Comparison of Lin–Kernighan initial tour and random tour in terms of solution quality.

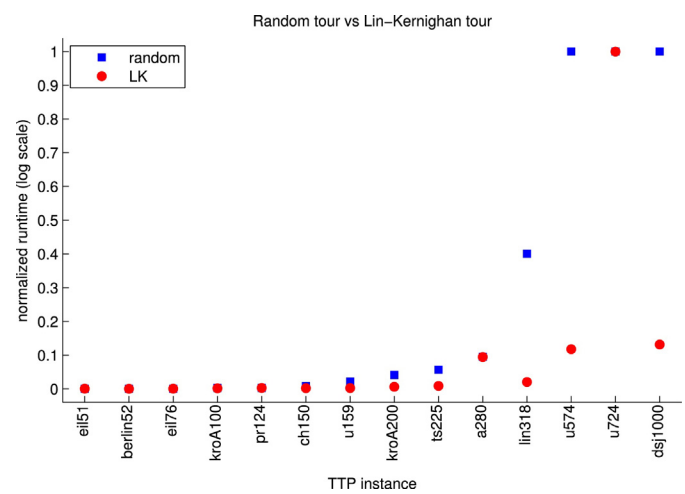

Fig. 5. Comparison of Lin–Kernighan initial tour and random tour in terms of runtime.

In Fig. 4, we illustrate the solution objective value of 14 different TTP instances of small and medium size. The used KP type is *bounded strongly correlated*, the knapsack category is 06, and the item factor is 3 [10]. Note that this experiment uses the JNB algorithm. Fig. 5 shows the same experiment in terms of runtime.

Note that the results shown are neither dependent on the used iterative algorithm, nor the TTP parameters (KP type, knapsack category, and item factor). We have omitted additional results for the sake of simplicity. In addition, we use a log scale to represent the runtimes in order to improve the plots visibility.

It is obvious that the Lin–Kernighan initialization technique is far better than the random tour method. As shown above, the results obtained starting with a Lin–Kernighan tour have far better objective values and runtimes.

4.1.2. Initial picking plan

To illustrate the influence of the starting picking plan, we use three initialization strategies: empty knapsack, random picking plan, and a greedy algorithm. These tests were performed using the same set of data presented above.

Figs. 6 and 7 illustrate the influence of the initial picking plan. As shown, clearly the initial picking plan has very insignificant influence on the solution quality and runtime. The standard deviation is 0.23% for objective values, and 16.25% for runtimes.

[3] Source codes for our two algorithms can be found in the website: https://github.com/yafrani/ttplab.

[4] All 9720 TTP instances and the raw results are availble at: https://sites.google.com/site/mohammadrezabonyadi/standarddatabases/travelling-thief-problem-data-bases-and-raw-results.

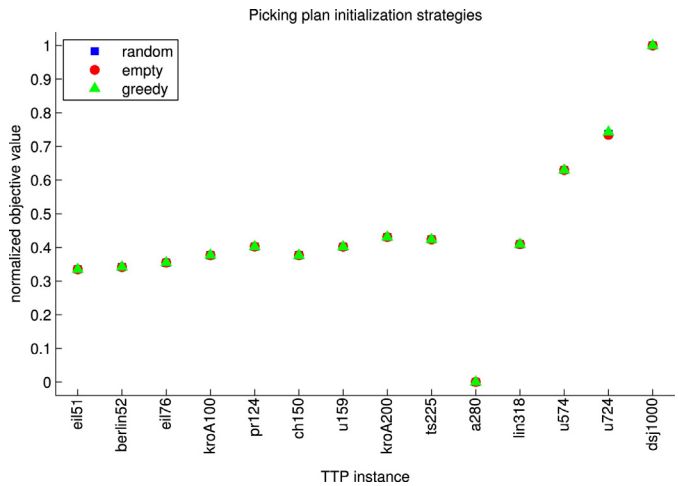

Fig. 6. Comparison between a random, an empty, and a greedy picking plan in terms of solution quality.

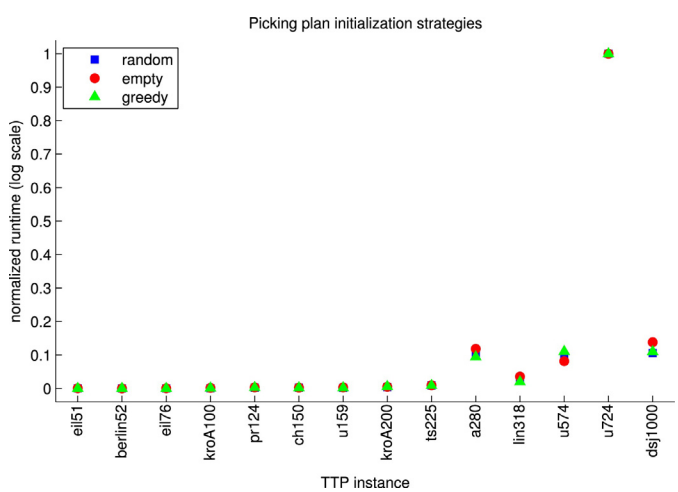

Fig. 7. Comparison between a random, an empty, and a greedy picking plan in terms of runtime.

4.1.3. Discussion

As shown above, the choice of the initial tour has a big influence on the search process. A decent TSP initial tour helps the iterative algorithm to find a good solution quickly. Whereas the initial picking plan have very slight influence on the solution quality and the runtime.

This behavior might be explained by noticing that the TTP instances used in this paper where generated using TSP instances from TSPLIB [32]. The fact that the data are strongly related to TSP instances might explain why a decent TSP tour gives a better solution in fewer iterations. This can also explain why TSP initial tour have more influence on the solving process than the initial picking plan.

4.2. Solving algorithms

In this subsection, our two algorithms are compared side-by-side with RLS and EA proposed in [10]. We perform tests on various TTP instances from the benchmark suite introduced in the same paper. These instances are selected by varying the TTP parameters as follows:

- **TSP base problems**: eil51, berlin52, eil76, kroA100, pr124, ch150, u159, kroA200, ts225, a280, lin318, u574, u724, dsj1000, rl1304
- **KP types (T)**: uncorrelated (U), uncorrelated with similar weights (USW), bounded strongly correlated (BSC)
- **Knapsack categories (C)**: 06, 10
- **Item factor (F)**: 1, 10

By combining these parameters, we obtain 180 TTP instances. Note that, the RLS and EA tests were performed on a different machine (Matlab 2014, core i7 2600 CPU 3.4 GHz, with 4 GB of RAM, running Windows 7). The runtimes for these algorithms are given for guidance. Also, due to the random behavior of these algorithms, 30 independent runs were performed. We select the best objective value for our comparison.

On the other hand, we use a Lin–Kernighan initial tour as it is the most efficient initialization strategy, and the greedy algorithm to initialize the picking plan. Since JNB and J2B are deterministic, one run per instance is adequate.

The results are presented in Tables 1–6. Note that the runtimes are measured with seconds, the best objective values are made bolder, and the negative ones are highlighted in gray.

We have made the following observations:

- JNB and J2B surpass RLS and EA objective values for many instances. They perform particularly well on small instances.
- JNB and J2B are far more time consuming than RLS and EA. Notably J2B is more time consuming than JNB. This is explained

Table 1
Results for the instances with 1 item per city and uncorrelated item weights/profits ($F = 1$, $T = \mathrm{U}$).

	Medium knapsack capacity ($C = 06$)								High knapsack capacity ($C = 10$)							
	EA		RLS		JNB		J2B		EA		RLS		JNB		J2B	
	obj	time	obj	time	obj	time	obj	time	obj	time	obj	time	obj	time	obj	time
eil51	3313	1.23	3313	0.85	**4744**	0.05	4604	0.25	6130	1.39	6130	0.78	**7481**	0.01	7446	0.05
berlin52	**5372**	1.25	**5372**	0.77	5342	0.02	5342	0.09	7980	1.09	7980	0.69	**8030**	0	**8030**	0.08
eil76	6176	0.99	6176	0.74	8286	0.01	**8519**	0.33	8826	0.93	8826	0.84	**11196**	0.01	**11196**	0.32
kroA100	9070	1.13	9070	1.02	10468	0.02	**10509**	2.19	14151	0.99	14151	0.75	**15017**	0.03	**15017**	1.45
pr124	**16767**	1.09	**16767**	1.37	15378	0.06	15360	1.09	18546	1.12	18546	0.99	17358	0.04	**19915**	3.86
ch150	14768	1.24	14768	0.8	15201	0.04	**15929**	19	20171	1.26	20171	0.9	20670	0.04	**21343**	21
u159	20546	1.14	20546	0.86	**20565**	0.05	**20565**	4.87	24885	1.13	24885	1.05	**24918**	0.08	**24918**	6.92
kroA200	**25450**	1.27	**25450**	0.95	22889	0.15	22889	6.39	**30673**	1.41	**30673**	1.05	28404	0.16	28408	6.37
ts225	**29451**	2.28	**29451**	0.91	26873	0.23	27025	49	**33949**	1.19	**33949**	0.89	31612	0.3	31721	54
a280	36140	1.44	36140	0.94	**36865**	1.87	36656	54	41358	1.41	41358	1.04	**42288**	2.5	35547	95
lin318	26310	1.87	26310	1.2	26479	0.46	**31058**	245	37573	1.76	37573	1.07	37764	0.69	**41489**	320
u574	67073	2.55	67073	1.27	**68361**	2.86	**68361**	600	88101	2.53	88101	1.53	**89499**	3.14	89497	600
u724	85406	2.91	85406	1.94	**86828**	7.94	86786	600	109681	2.95	109681	1.93	**111158**	16	110995	600
dsj1000	76040	5.29	76040	2.09	**81296**	12	**81296**	600	150403	5.29	150403	2.16	**157383**	11	**157383**	600
rl1304	**148740**	7.25	**148740**	3.72	146608	149	144970	600	**205093**	8	**205093**	3.25	203860	189	200213	600

Table 2
Results for the instances with 1 item per city and uncorrelated but similar item weights ($F = 1$, $T = \text{USW}$).

	Medium knapsack capacity ($C = 06$)								High knapsack capacity ($C = 10$)							
	EA		RLS		JNB		J2B		EA		RLS		JNB		J2B	
	obj	time	obj	time	obj	time	obj	time	obj	time	obj	time	obj	time	obj	time
eil51	1933	0.79	1933	0.79	**2929**	0	**2929**	0.08	5420	1	5420	0.85	**6519**	0.01	**6519**	0.04
berlin52	3848	1.12	3848	0.81	**3856**	0	**3856**	0.05	7232	1	7232	0.69	**7297**	0	**7297**	0.07
eil76	1602	1.67	1602	0.81	**3993**	0.01	**3993**	0.37	7808	1.23	7808	0.79	**10315**	0.01	**10315**	0.35
kroA100	**9925**	1.39	**9925**	0.75	9481	0.02	9514	1.08	13880	1.04	13880	1.03	**13911**	0.03	**13926**	1.09
pr124	**12852**	1.14	**12852**	0.97	10733	0.05	12244	4.6	17943	1.13	17943	0.97	15806	0.07	**18941**	4.28
ch150	12112	1.47	12112	0.94	12590	0.04	**13470**	24	18446	1.18	18446	0.79	**19023**	0.05	**19023**	4.77
u159	15223	1.67	15223	0.81	**15243**	0.08	**15243**	6.79	22772	1.13	22772	1.08	**22803**	0.06	**22803**	4.92
kroA200	**19921**	1.19	**19921**	1.08	17358	0.14	18581	14	**27802**	1.18	**27802**	1.01	25041	0.17	25041	12
ts225	**18562**	1.29	**18562**	0.97	17210	0.32	17210	58	**31365**	1.45	**31365**	0.98	30895	0.31	30895	32
a280	22469	1.37	22468	1.03	**23363**	2.01	22896	245	38789	1.54	38789	0.97	**39859**	2.4	29142	184
lin318	16285	1.49	16285	1.08	16517	0.94	**22122**	258	33797	1.55	33797	1.11	34057	0.61	**36909**	583
u574	52481	2.62	52481	1.4	**53711**	2.97	**53711**	600	79541	2.47	79541	1.8	**81008**	1.95	**81008**	427
u724	59902	3.59	59902	1.9	**61964**	14	61934	600	100330	3.15	100330	1.56	**102892**	21	102713	600
dsj1000	41167	6.35	41167	2.6	**47885**	13	**47885**	600	149660	5.33	149660	2.28	**155635**	13	**155635**	600
rl1304	**124420**	6.83	**124420**	4.14	119749	98	117341	600	**190956**	6.91	**190956**	2.98	188811	197	185209	600

Table 3
Results for the instances with 1 item per city and bounded strongly correlated item weights/profits ($F = 1$, $T = \text{BSC}$).

	Medium knapsack capacity ($C = 06$)								High knapsack capacity ($C = 10$)							
	EA		RLS		JNB		J2B		EA		RLS		JNB		J2B	
	obj	time	obj	time	obj	time	obj	time	obj	time	obj	time	obj	time	obj	time
eil51	5899	0.94	5899	0.79	6649	0.01	**6773**	0.37	9613	1.04	9613	0.88	11439	0.01	**11750**	0.11
berlin52	7900	1.04	7900	0.75	7537	0	**11896**	0.45	7875	0.86	7875	0.78	7993	0	**13107**	0.73
eil76	9961	1.73	9961	0.81	**12215**	0.01	**12215**	0.43	10033	1.06	10033	0.73	**14862**	0.01	**14862**	0.58
kroA100	14759	1.01	14747	0.85	18652	0.02	**18731**	1.35	14762	1.02	14762	0.84	**20219**	0.03	**20219**	1.66
pr124	16410	1.79	16410	0.9	25081	0.12	**25122**	1.44	21770	1.4	21770	0.88	27247	0.05	**27277**	1.46
ch150	17915	1.28	17915	0.99	18721	0.07	**19070**	17	17149	1.44	17149	0.81	**18422**	0.09	**18422**	9.64
u159	34121	1.16	34119	0.86	**34134**	0.1	**34134**	11	37359	1.22	37359	0.93	37448	0.12	**39131**	19
kroA200	**43307**	1.25	**43307**	1.04	40746	0.22	40660	9.08	**39016**	1.44	**39016**	1.22	38155	0.3	38179	15
ts225	47606	1.21	47606	0.99	**48556**	0.55	**48556**	109	48921	1.29	48921	0.9	**51833**	0.34	**51833**	36
a280	44995	2.3	44989	1.09	**46478**	4.33	42780	486	51302	1.45	51302	1.03	53828	4.46	**58121**	600
lin318	37323	4.12	37298	1.07	37225	1.21	**56344**	600	42959	2.4	42959	1.22	43371	1.17	**56679**	600
u574	110444	2.58	110444	1.31	**111935**	2.73	**111935**	600	131604	2.38	131604	1.41	**136156**	3.26	**136156**	603
u724	**136892**	3.44	136890	1.83	133736	25	133038	600	**148256**	3.55	**148256**	1.7	146040	47	144040	600
dsj1000	328978	6.4	328979	2.37	**332572**	14	332544	600	293069	6.37	293069	2.6	**306279**	11	**306279**	600
rl1304	291699	8.23	**291700**	3.51	263521	342	254820	600	**303556**	6.84	**303556**	4.66	282758	368	268560	600

Table 4
Results for the instances with 10 items per city and uncorrelated item weights/profits ($F = 10$, $T = \text{U}$).

	Medium knapsack capacity (C=06)								High knapsack capacity (C=10)							
	EA		RLS		JNB		J2B		EA		RLS		JNB		J2B	
	obj	time	obj	time	obj	time	obj	time	obj	time	obj	time	obj	time	obj	time
eil51	44624	2.63	44624	1.08	**55061**	1.62	54986	0.86	62588	1.8	62588	1	73734	1.49	**73763**	24
berlin52	67241	1.8	67241	0.94	**67690**	0.67	**67690**	31	86050	1.87	86050	1.08	**86543**	0.66	**86543**	32
eil76	58616	2.81	58616	1.2	**78400**	1.75	**78400**	124	85545	2.67	85545	1.14	**107858**	2.28	**107858**	161
kroA100	107974	3.14	107974	1.5	115983	4.76	**118105**	163	148491	3.14	148491	1.43	150959	4.19	**156239**	241
pr124	142013	4.13	142013	1.96	139648	7.4	**143382**	11	185933	4.53	185933	2.17	180553	9.03	**186667**	10
ch150	126962	6.33	126962	1.96	**131701**	16	126730	600	188935	5.82	188935	1.82	**194376**	17	188093	600
u159	188111	5.28	188111	2.31	**188404**	11	178250	603	242260	6.24	242260	2.29	**242539**	15	222626	600
kroA200	225584	8.81	225584	3.02	226805	41	**227358**	65.5	294118	9.93	294118	2.7	296686	41	**297184**	66
ts225	**264424**	9.87	**264424**	3.49	262014	32	139575	600	**343983**	12	**343983**	2.93	339645	43	186545	600
a280	**305734**	17	**305734**	4.2	70435	500	12371	600	**411715**	17	**411715**	3.66	330510	435	27373	600
lin318	**275033**	26	**275033**	4.64	258410	71	125336	600	**398115**	20	**398115**	5.21	371790	82	-14516	600
u574	681934	69	681934	15	**693160**	600	692634	600	924203	71	924203	13	937133	600	**936234**	600
u724	**863165**	104	**863165**	17	810622	600	790940	599	**1157344**	98	**1157344**	17	1045846	600	1025704	600
dsj1000	1161151	182	1161151	35	**1190676**	582	1190177	600	1411451	188	1411451	39	**1459170**	600	1458413	600
rl1304	**1666972**	319	**1666972**	72	1154690	600	1117255	600	**2183617**	275	**2183617**	67	1558021	600	1514964	600

by noticing that J2B's neighborhood has a higher complexity than JNB.

- JNB and J2B perform better for small instances.

  Nevertheless, their performance decreases substantially for mid-size and large instances. The reason for this is that our heuristics are local search algorithms designed to better exploit the search space instead of exploring. As in small instances the search space is smaller, our algorithms find easily a decent solution. For large instances, two scenarios can occur:

  1. Premature convergence, i.e. the algorithm gets stuck in a local optimum.
  2. The algorithm takes a lot of time browsing the generated neighborhoods. Thus, it is stopped (e.g. most instances with a runtime of 600 s).

Table 5
Results for the instances with 10 items per city and uncorrelated but similar item weights ($F = 10$, $T = \text{USW}$).

	Medium knapsack capacity (C=06)								High knapsack capacity (C=10)							
	EA		RLS		JNB		J2B		EA		RLS		JNB		J2B	
	obj	time	obj	time	obj	time	obj	time	obj	time	obj	time	obj	time	obj	time
eil51	30880	1.59	30880	1.4	40851	2.37	**41248**	0.78	52794	2.44	52794	1.06	**65425**	2.08	**65425**	84
berlin52	46584	1.72	46584	0.98	**46998**	1.23	**46998**	57	79072	1.77	79072	0.99	**79530**	1.37	**79530**	66
eil76	41667	2.58	41667	1.21	**63035**	3.33	**63035**	242	77510	2.54	77510	1.31	**101126**	2.71	**101126**	193
kroA100	88765	2.81	88765	1.87	88561	9.11	**93555**	396	133914	3.04	133914	1.71	136263	6.46	**141826**	401
pr124	**107625**	4.8	**107625**	2.47	103652	21	23640	600	**172977**	4.57	**172977**	1.81	162537	17	125860	600
ch150	91650	7.37	91650	1.75	**96605**	21	90455	600	170256	4.78	170256	1.82	**176079**	17	169410	600
u159	128974	6.32	128974	2.95	**129279**	19	77204	600	217592	8.14	217592	2.71	**217931**	15	188954	600
kroA200	**184058**	9.21	**184058**	2.43	176746	41	177166	68	**271192**	9.58	**271192**	2.63	267436	49	268353	66
ts225	**188464**	11	**188464**	2.8	186519	56	-156846	600	308174	10	308174	3.18	**310089**	64	3186	600
a280	**229256**	21	**229256**	3.94	53420	600	17861	600	**375868**	16	**375868**	3.91	249661	600	35593	600
lin318	**185680**	22	**185680**	6.37	162389	210	-142068	600	**333455**	21	**333455**	4.92	310881	161	-306403	600
u574	521313	60	521313	15	**533296**	566	532881	600	816095	53	816095	13	**829432**	600	828576	600
u724	**655029**	93	**655029**	21	507367	600	473236	600	**1023238**	86	**1023238**	17	861585	600	828037	600
dsj1000	722132	205	722132	39	**757347**	600	757041	600	1238041	178	1238041	46	**1284257**	600	1283538	600
rl1304	**1273882**	347	**1273882**	73	-113919	600	-177210	600	**1957232**	303	**1957232**	60	822586	600	776959	600

Table 6
Results for the instances with 10 items per city and bounded strongly correlated item weights/profits ($F = 10$, $T = \text{BSC}$).

	Medium knapsack capacity (C=06)								High knapsack capacity (C=10)							
	EA		RLS		JNB		J2B		EA		RLS		JNB		J2B	
	obj	time	obj	time	obj	time	obj	time	obj	time	obj	time	obj	time	obj	time
eil51	78874	1.76	78872	1.04	**94337**	2.16	94200	79	75884	2.18	75884	1.3	100455	0.9	**102066**	0.33
berlin52	113112	1.69	113096	1.04	**113818**	1.38	**113818**	68	117286	1.75	117286	1.03	118167	0.79	**137949**	45
eil76	100554	3.22	100556	1.69	**138159**	4.78	**138159**	362	102447	2.52	102447	1.6	**159321**	3.08	**159321**	224
kroA100	169894	4.76	169894	1.72	186547	6.51	**204392**	393	167127	4.63	167127	1.6	193124	2.13	**207846**	265
pr124	230814	4.77	230814	1.77	240058	12	**258776**	10	253586	5.89	253586	1.56	251613	4.11	**273036**	10
ch150	248409	7.04	248406	2.36	**256254**	58	233334	600	211512	6.99	211512	2.32	**224417**	48	201398	600
u159	287344	8.03	287346	2.85	**287881**	28	240528	600	316843	6.2	316843	2.32	**317585**	21	283721	600
kroA200	**382166**	13	**382166**	3.9	358811	102	360889	66	**416638**	9.88	**416638**	3.03	389017	75	392880	64
ts225	**446400**	13	**446400**	3.43	438750	62	235789	600	485578	14	485578	3.52	**495835**	40	336704	600
a280	**485872**	19	**485872**	5.22	388762	600	91306	600	**581075**	18	**581075**	4.95	570327	600	91587	600
lin318	388663	24	**388664**	5.25	319823	92	160915	600	**364584**	20	**364584**	6.02	284813	49	-4887	600
u574	1038239	63	1038239	16	**1052371**	600	1051570	600	1249081	60	1249081	13	**1275014**	600	1271965	600
u724	**1397129**	132	**1397129**	28	1127709	599	1096003	600	**1535001**	102	**1535001**	27	1360216	600	1323516	600
dsj1000	1697525	172	**1697527**	41	1695939	600	1695168	600	1877107	197	1877107	49	**1954051**	600	1951910	600
rl1304	**2804854**	315	**2804854**	74	1713492	600	1636605	599	**3076322**	400	**3076322**	83	2352288	600	2263461	600

Note that we retested the instances with negative gain without a runtime limit and obtained better results. This implies that the reason for obtaining a negative value for these instances is due to the runtime limit.

- JNB has a better objective/runtime ratio for larger TTP instances, while J2B has better a solution quality for small TTP instances.

5. Conclusion and future work

In this paper, we study the Travelling Thief Problem which is an NP-hard multi-component problem. Firstly, we conduct a theoretical study in which we formulate the problem mathematically. The paper focuses on the interdependence between the TSP and KP sub-problems, which is the source of hardness of the TTP. We show that it is inefficient to solve sub-problems in isolation from each other. Therefore, we prove that it is impossible to recover the objective value of a mutated solution in a constant time. Then, a simple approach that consists of the combination of sub-problems neighborhoods is introduced. Two simple algorithms that implement this approach are proposed. These algorithms are compared in terms of solution quality and runtime alongside with two other algorithms from [10]. The tests revealed that our algorithms found solutions that surpass EA and RLS in terms of objective value for many TTP instances. The proposed heuristics perform well on small TTP instances. However, their performance decreases significantly for larger instances due to the time complexity. Nevertheless, the main purpose of these algorithms is to provide a starting point for future investigations on neighborhood based heuristics.

We believe that our approach is important for these main reasons:

1. Neighborhood-based algorithms are the essence of other more efficient algorithms such as Tabu Search [33,34], Simulated Annealing [29], etc.
2. Our algorithms are specially designed to exploit particular areas in the search space efficiently.
3. Our approach is not limited to the TTP, but could be extended to other multi-component problems.
4. The proposed backup technique for mutated solutions can be used in other algorithms to reduce the complexity of the objective function.

In the future, we engage to improve the performance of our algorithms and provide more sophisticated heuristics able to solve large TTP instances. For this purpose, we aim to use a better items picking strategy than the naive method proposed in this paper. We also plan to provide better neighborhood methods in terms of runtime (e.g. neighborhood reduction techniques) and search space exploitation, and provide a more efficient algorithm based on our approach. Furthermore, we intend to propose a memetic algorithm to combine local search with other evolutionary algorithms in order to avoid premature convergence.

References

[1] J. Van Leeuwen, J. Leeuwen, Handbook of Theoretical Computer Science: Algorithms and Complexity, vol. 1, Elsevier, 1990.

[2] S. Martello, P. Toth, Knapsack Problems, Wiley, New York, 1990.
[3] M.M. Flood, The traveling-salesman problem, Oper. Res. 4 (1) (1956) 61–75.
[4] T.R. Jensen, B. Toft, Graph Coloring Problems, vol. 39, John Wiley & Sons, 2011.
[5] F. Glover, G.A. Kochenberger, Handbook of Metaheuristics, Springer, 2003.
[6] Z. Michalewicz, Quo vadis, evolutionary computation? in: Advances in Computational Intelligence, Springer, 2012, pp. 98–121.
[7] Z. Michalewicz, Ubiquity Symposium: Evolutionary Computation and the Processes of Life: The Emperor Is Naked: Evolutionary Algorithms for Real-World Applications, Ubiquity 2012, November 2012, p. 3.
[8] M.R. Bonyadi, Z. Michalewicz, L. Barone, The travelling thief problem: the first step in the transition from theoretical problems to realistic problems, in: 2013 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2013, pp. 1037–1044.
[9] J. Perl, M.S. Daskin, A warehouse location-routing problem Transp. Res. Part B: Methodol. 19 (5) (1985) 381–396.
[10] S. Polyakovskiy, M. Reza, M. Wagner, Z. Michalewicz, F. Neumann, A comprehensive benchmark set and heuristics for the traveling thief problem, in: Proceedings of the Genetic and Evolutionary Computation Conference (GECCO), Vancouver, Canada, 2014.
[11] M.R. Bonyadi, Z. Michalewicz, M. Roman Przybyŏek, A. Wierzbicki, Socially inspired algorithms for the travelling thief problem, in: Proceedings of the 2014 Conference on Genetic and Evolutionary Computation, ACM, 2014, pp. 421–428.
[12] Y. Mei, X. Li, X. Yao, On investigation of interdependence between sub-problems of the travelling thief problem, Soft Comput. (2014) 1–16.
[13] Y. Mei, X. Li, X. Yao, Improving efficiency of heuristics for the large scale traveling thief problem, in: Simulated Evolution and Learning, Springer, 2014, pp. 631–643.
[14] M. Ibrahimov, A. Mohais, S. Schellenberg, Z. Michalewicz, Evolutionary approaches for supply chain optimisation. Part I: Single and two-component supply chains, Int. J. Intell. Comput. Cybern. 5 (4) (2012) 444–472.
[15] M. Ibrahimov, A. Mohais, S. Schellenberg, Z. Michalewicz, Evolutionary approaches for supply chain optimisation. Part II: Multi-silo supply chains, Int. J. Intell. Comput. Cybern. 5 (4) (2012) 473–499.
[16] M.T. Melo, S. Nickel, F. Saldanha-da Gama, Facility location and supply chain management – a review, Eur. J. Oper. Res. 196 (2) (2009) 401–412.
[17] N. Shah, Pharmaceutical supply chains: key issues and strategies for optimisation, Comput. Chem. Eng. 28 (6) (2004) 929–941.
[18] T. Santoso, S. Ahmed, M. Goetschalckx, A. Shapiro, A stochastic programming approach for supply chain network design under uncertainty, Eur. J. Oper. Res. 167 (1) (2005) 96–115.
[19] M. Iori, S. Martello, Routing problems with loading constraints, Top 18 (1) (2010) 4–27.
[20] J. Stolk, I. Mann, A. Mohais, Z. Michalewicz, Combining vehicle routing and packing for optimal delivery schedules of water tanks, OR Insight 26 (3) (2013) 167–190.
[21] Y. Takayuki, S. Takahiro, Complexity and completeness of finding another solution and its application to puzzles, IEICE Trans. Fundam. Electron. Commun. Comput. Sci. 86 (5) (2003) 1052–1060.
[22] O. Ruepp, M. Holzer, The computational complexity of the Kakuro puzzle, revisited, in: Fun with Algorithms, Springer, 2010, pp. 319–330.
[23] Y. Marinakis, An improved particle swarm optimization algorithm for the capacitated location routing problem and for the location routing problem with stochastic demands, Appl. Soft Comput. 37 (2015) 680–701.
[24] F.Y. Vincent, S.-W. Lin, Multi-start simulated annealing heuristic for the location routing problem with simultaneous pickup and delivery, Appl. Soft Comput. 24 (2014) 284–290.
[25] S.C. Leung, Z. Zhang, D. Zhang, X. Hua, M.K. Lim, A meta-heuristic algorithm for heterogeneous fleet vehicle routing problems with two-dimensional loading constraints, Eur. J. Oper. Res. 225 (2) (2013) 199–210.
[26] M. Gendreau, M. Iori, G. Laporte, S. Martello, A tabu search algorithm for a routing and container loading problem, Transp. Sci. 40 (3) (2006) 342–350.
[27] G. Fuellerer, K.F. Doerner, R.F. Hartl, M. Iori, Ant colony optimization for the two-dimensional loading vehicle routing problem, Comput. Oper. Res. 36 (3) (2009) 655–673.
[28] G. Croes, A method for solving traveling-salesman problems, Oper. Res. 6 (6) (1958) 791–812.
[29] S. Kirkpatrick, C.D. Gelatt, M.P. Vecchi, et al., Optimization by simulated annealing, Science 220 (4598) (1983) 671–680.
[30] S. Lin, B.W. Kernighan, An effective heuristic algorithm for the traveling-salesman problem, Oper. Res. 21 (2) (1973) 498–516.
[31] G. Reinelt, Fast heuristics for large geometric traveling salesman problems, ORSA J. Comput. 4 (2) (1992) 206–217.
[32] G. Reinelt, Tsplib – a traveling salesman problem library, ORSA J. Comput. 3 (4) (1991) 376–384.
[33] F. Glover, Tabu search – Part I, ORSA J. Comput. 1 (3) (1989) 190–206.
[34] F. Glover, Tabu search – Part II, ORSA J. Comput. 2 (1) (1990) 4–32.